

FUNDAMENTOS, TIPOS & CONTROLE



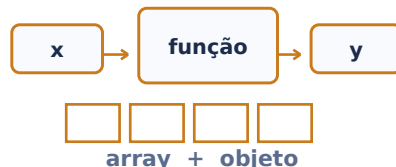
dados → decisão → repetição

Dominar a sintaxe essencial, tipos de dados e fluxo de execução antes de construir aplicações maiores.

- **Declaração:** const para referência fixa; let para valor mutável
- **Tipos:** string, number, boolean, undefined, null, bigint, symbol
- **Comparação:** prefira === e !== para evitar coerção implícita
- **Controle:** if/else, switch e operador ternário
- **Laços:** for, while, for...of; use break/continue quando necessário
- **Escopo:** let/const respeitam bloco; var possui escopo de função

Dica: Use const por padrão e mude para let somente quando precisar reatribuir. Prefira === para reduzir comportamentos inesperados.

FUNÇÕES, ARRAYS & OBJETOS



Organizar comportamento e dados com funções reutilizáveis, coleções e objetos.

- **Função:** parâmetros → processamento → retorno
- **Arrow:** (x) => x*2; não possui seu próprio this
- **Array:** push/pop; map, filter, find, reduce e forEach
- **Objeto:** pares chave: valor; acesso obj.campo ou obj["campo"]
- **Desestruturação:** const {nome} = pessoa; const [a,b] = lista
- **Spread:** {...obj} e [...arr] criam cópias rasas/combinções
- **Rest:** (...args) agrupa argumentos restantes

Dica: map transforma, filter seleciona e reduce acumula. Escolher o método certo deixa o código menor e mais legível.

DOM, EVENTOS & ASSÍNCRONO



Conectar JavaScript à interface do navegador e lidar com tarefas que terminam no futuro.

- **DOM:** document.querySelector() localiza elementos
- **Conteúdo:** textContent; classes via classList; atributos via setAttribute
- **Eventos:** addEventListener("click", callback)
- **Event:** target identifica origem; preventDefault() cancela comportamento padrão
- **Promise:** representa operação pendente, resolvida ou rejeitada
- **async/await:** sintaxe legível sobre Promises
- **Fetch:** const r = await

Dica: Não bloqueie a interface esperando rede ou timer. Use async/await e sempre trate erros de requisições externas.

ES6+, MÓDULOS & BOAS PRÁTICAS



export • import • testes

Estruturar projetos modernos com módulos, classes, tratamento de erros e código fácil de manter.

- **Módulos:** export / import separam responsabilidades
- **Classes:** constructor, métodos, extends e super
- **Optional chaining:** obj?.perfil?.nome evita acesso inválido
- **Nullish:** valor ?? padrão usa fallback só para null/undefined
- **Template string:** `Olá \${nome}` para interpolação
- **JSON:** JSON.stringify() e JSON.parse()
- **Qualidade:** funções pequenas, nomes claros, testes e linting

Dica: Separe UI, regra de negócio e acesso a dados. Módulos pequenos e previsíveis tornam testes, manutenção e evolução muito mais simples.